

the range and moves to the right end ❷. We compare the item at index j with the pivot value and then move right to compare the next item. The pivot value can be arbitrarily chosen from any value in the range, but we'll always use the value at the right end of the range.

Imagine a second index i that also begins at the left end. If the item at index j is less than or equal to the pivot, the items at indices i and j are swapped ❸ and i is increased to the next index. So while j always increases (that is, moves right) after each comparison with the pivot value, i increases only if the item at index j is less than or equal to the pivot.

The names i and j are commonly used for variables that hold array indices. Someone else's `quicksort()` implementation may instead use j and i , or even completely different variables. The important thing to remember is that two variables store indices and behave as shown here.

As an example, let's work through the first partitioning of the array `[0, 7, 6, 3, 1, 2, 5, 4]`, and the range defined by `left` of 0 and `right` of 7 to cover the full size of the array. The `pivot` will be the value at the `right` end, 4. The i and j index begin at index 0, the left end of the range. At each step, index j always moves to the right. Index i moves only if the value at index j is less than or equal to the pivot value. The `items` array, the i index, and the j index begin as follows:

items:	[0, 7, 6, 3, 1, 2, 5, 4]
indices:	0 1 2 3 4 5 6 7
	^
$i = 0$	i
$j = 0$	j

The value at index j (which is 0) is less than or equal to the pivot value (which is 4), so swap the values at i and j . This results in no actual change since i and j are the same index. Also, increase i so that it moves to the right. The j index increases for every comparison with the pivot value. The state of the variables now looks like this:

items:	[0, 7, 6, 3, 1, 2, 5, 4]
indices:	0 1 2 3 4 5 6 7
	^
